



UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

PROGRAMACIÓN 2:

TRABAJO PRÁCTICO INTEGRADOR

Grupo: 9

Estudiantes:

- Fiorella Salazar - Comisión 10
- Virginia Laura Paloma - Comisión 10
- Jonathan Soza - Comisión 10
- Facundo Villarroel - Comisión 5

Fecha de entrega: 22/06/2026

Índice

Marco Teórico.....	3
- Java:.....	3
- Programación estructurada:.....	3
- Programación orientada a objetos (POO):.....	4
- UML:.....	4
- Colecciones:.....	4
- Herencia y polimorfismo:.....	5
- Interfaces:.....	5
- Excepciones:.....	5
- Genéricos:.....	6
Decisiones Técnicas y Arquitectura.....	7
Dificultades y Soluciones.....	13
Bibliografía / Webgrafía.....	15
Enlaces.....	16

Marco Teórico

- Java:

Java es un lenguaje de programación orientado a objetos ampliamente utilizado para el desarrollo de aplicaciones debido a su simplicidad, portabilidad y robustez.

Durante el desarrollo de este trabajo se utilizó Java como lenguaje principal debido a que permite aplicar de forma práctica los conceptos fundamentales de programación orientada a objetos y organizar el código de manera clara y mantenible.

- Programación estructurada:

La programación estructurada constituye la base para el desarrollo de aplicaciones y se centra en organizar el código mediante instrucciones, condiciones y repeticiones. Para ello, utiliza estructuras condicionales que permiten tomar decisiones según determinadas situaciones y ciclos que facilitan la ejecución repetitiva de tareas.

Además, emplea funciones o métodos para dividir el programa en partes más pequeñas y reutilizables, mejorando la organización y legibilidad del código. También hace uso de estructuras de datos como los arrays para almacenar y gestionar conjuntos de información.

A continuación, se detallan qué conceptos fueron utilizados durante el desarrollo del trabajo para controlar el flujo de ejecución del sistema, validar datos ingresados por el usuario y gestionar la información almacenada en memoria.

- Las estructuras condicionales permiten que un programa tome diferentes decisiones según determinadas condiciones. A través de instrucciones como if, else y switch es posible controlar el flujo de ejecución y adaptar el comportamiento del sistema a distintas situaciones.
- Los ciclos o estructuras repetitivas permiten ejecutar un bloque de código varias veces mientras se cumpla una condición o durante una cantidad determinada de iteraciones. Entre las estructuras más utilizadas se encuentran for, while y do-while, las cuales resultan esenciales para automatizar tareas repetitivas y procesar conjuntos de datos.
- Las funciones o métodos también constituyen una herramienta fundamental ya que permiten dividir un problema complejo en tareas más pequeñas y reutilizables. Gracias a esto el código se vuelve más modular, legible y fácil de mantener, además de reducir la duplicación de instrucciones.

- Los arreglos o arrays son de tamaño fijo y permiten almacenar múltiples valores de un mismo tipo de dato dentro de una única estructura, son muy útiles para organizar información relacionada y facilitar su acceso mediante índices. Aunque posteriormente fueron complementados por estructuras más flexibles como las colecciones, de las cuales hablaremos más adelante. Los arrays continúan siendo una herramienta importante para comprender la gestión de datos en programación.

- Programación orientada a objetos (POO):

La Programación Orientada a Objetos es un paradigma de programación que se basa en la representación de los elementos de un sistema mediante objetos. Cada objeto posee atributos, que representan sus características, y métodos, que representan sus comportamientos.

Este paradigma permite modelar situaciones del mundo real de una manera más natural y facilita la reutilización y el mantenimiento del código. Algunos de los conceptos más importantes son las clases, la encapsulación, la abstracción, la herencia y el polimorfismo. Gracias a estos conceptos es posible desarrollar aplicaciones más organizadas y escalables.

En este trabajo, la Programación Orientada a Objetos fue fundamental para modelar las distintas entidades del sistema como Usuarios, Productos, Categorías y Pedidos. Cada una de ellas fue representada mediante clases que contienen los datos y comportamientos necesarios para su funcionamiento, permitiendo una mejor organización del código y una estructura más fácil de mantener y ampliar.

- UML:

El Lenguaje Unificado de Modelado (UML) es una herramienta utilizada para representar gráficamente la estructura y el comportamiento de un sistema antes de su implementación. Su utilización permite comprender mejor los distintos componentes de una aplicación y las relaciones existentes entre ellos.

En este trabajo se nos proporcionó un diagrama UML sobre el cual debíamos basarnos para desarrollar las clases del sistema, definiendo sus atributos, métodos y relaciones. Esto permitió visualizar de forma anticipada la organización general del proyecto y sirvió como guía durante el desarrollo de la aplicación.

- Colecciones:

Las colecciones son estructuras de datos que permiten almacenar y administrar conjuntos de objetos de manera dinámica. Java proporciona distintas implementaciones a través del Framework de Colecciones, entre las que se destacan las listas, conjuntos y mapas.

El uso de colecciones resulta especialmente útil cuando se trabaja con cantidades variables de datos ya que permiten agregar, eliminar, buscar y recorrer elementos de manera sencilla. En este proyecto se utilizaron para almacenar y gestionar las instancias de las clases propuestas en el UML (Categoría, Producto, Usuarios y Pedidos), necesarias para el funcionamiento del sistema sin necesidad de recurrir a estructuras estáticas.

Además, las colecciones facilitaron la manipulación de la información durante la ejecución del programa, permitiendo realizar operaciones de búsqueda, listado y actualización de datos de forma más flexible y organizada.

- Herencia y polimorfismo:

La herencia es un mecanismo que permite que una clase pueda adquirir atributos y métodos de otra clase. Esto favorece la reutilización de código y permite establecer relaciones jerárquicas entre distintos tipos de objetos.

En este proyecto se utilizó una clase Base común de la cual heredan las distintas entidades del sistema. Esto permitió centralizar atributos y comportamientos compartidos, evitando la duplicación de código y facilitando el mantenimiento de la aplicación.

Por otra parte, el polimorfismo permite que diferentes objetos puedan responder de manera distinta a una misma operación según su implementación específica. Gracias a estos conceptos es posible desarrollar soluciones más flexibles y reducir la duplicación de código. Su aplicación facilita la extensión del sistema y mejora la organización general del diseño.

- Interfaces:

Las interfaces permiten definir un conjunto de métodos que una clase debe implementar sin especificar cómo se desarrollará internamente cada uno de ellos. De esta forma se establece un contrato que garantiza determinados comportamientos independientemente de la implementación concreta.

El uso de interfaces favorece el desacoplamiento entre componentes y mejora la flexibilidad del sistema. Además, permiten diseñar aplicaciones más modulares y facilitan futuras modificaciones o ampliaciones sin afectar el funcionamiento general del programa.

- Excepciones:

Las excepciones son mecanismos que permiten detectar y gestionar errores que pueden producirse durante la ejecución de un programa. Su utilización evita que la aplicación finalice de manera inesperada y permite brindar una respuesta

adecuada ante situaciones imprevistas.

Java nos proporciona un sistema de manejo de excepciones mediante las estructuras try, catch y finally, las cuales permiten capturar errores y tratarlos de forma controlada. En este trabajo se aplicó este mecanismo para validar datos de entrada y mejorar la confiabilidad del sistema y de los datos, garantizando una experiencia de uso más segura y estable.

- Genéricos:

Los Genéricos son una característica de Java que permite definir clases, interfaces y métodos capaces de trabajar con distintos tipos de datos de manera segura y reutilizable. Mediante el uso de parámetros de tipo, es posible especificar qué tipo de objeto se utilizará en una estructura o método, evitando errores de conversión y mejorando el control de tipos durante la compilación.

Su principal objetivo es aumentar la flexibilidad del código sin perder seguridad, permitiendo reutilizar una misma implementación para diferentes tipos de datos. Además, facilitan la legibilidad y el mantenimiento de las aplicaciones al hacer explícito el tipo de información que se manipula.

En este trabajo, los genéricos fueron utilizados en el paquete Utils, específicamente en la clase Validaciones, mediante el método *filtrarActivos()*. Este método genérico permite recibir y procesar listas de distintas entidades que heredan de la clase Base, retornando únicamente los objetos que no se encuentran eliminados, es decir, aquellos cuyo atributo eliminado posee el valor false. Gracias a esta implementación fue posible reutilizar la misma lógica en los distintos servicios del sistema, evitando la duplicación de código y mejorando la organización y mantenibilidad de la aplicación.

Decisiones Técnicas y Arquitectura

Para el desarrollo del sistema se optó por una organización modular del proyecto, separando las responsabilidades en diferentes paquetes con el objetivo de mejorar la legibilidad del código, facilitar el mantenimiento y favorecer la reutilización de componentes.

El punto de entrada de la aplicación es la clase Main, encargada de mostrar el menú principal y gestionar la interacción inicial con el usuario. Desde este menú se accede a distintos submenús específicos para cada módulo del sistema: Categorías, Productos, Usuarios y Pedidos. Esta decisión permitió evitar que toda la lógica de navegación estuviera concentrada en una única clase y facilitó la organización de las funcionalidades.

Los submenús actúan como intermediarios entre la interfaz de usuario y la lógica de la aplicación. Su responsabilidad principal consiste en solicitar datos al usuario, mostrar información por consola y delegar las operaciones correspondientes a las clases de servicio.

La lógica de negocio se encuentra centralizada en las clases del paquete services. Estas clases contienen métodos estáticos responsables de realizar operaciones sobre las entidades y las colecciones almacenadas en memoria. Entre sus responsabilidades se incluyen las operaciones CRUD (Crear, Leer, Modificar y Eliminar) y validación de datos. Esta separación permitió mantener los submenús enfocados únicamente en la interacción con el usuario y evitar que la lógica del sistema y reglas de negocio estuvieran distribuidas entre las distintas capas de la aplicación.

Para almacenar la información del sistema en memoria se creó la clase Data dentro del paquete data. Esta clase contiene colecciones estáticas que mantienen en memoria las instancias de las entidades principales del sistema. De esta manera se centraliza el acceso a los datos y se evita la duplicación de estructuras de almacenamiento en distintas partes del programa.

Las entidades del sistema se encuentran agrupadas dentro del paquete entities. Todas ellas heredan de una clase base común denominada Base, la cual concentra atributos y comportamientos compartidos. La utilización de herencia permitió reducir la repetición de código y mantener una estructura más uniforme entre las distintas entidades definidas en el modelo UML.

Los conjuntos específicos de opciones se implementaron mediante enumeraciones (enums) ubicadas en el paquete enums. Esta decisión permitió representar los estados, formas de pago y roles de forma más segura y controlada, evitando el uso de valores arbitrarios dentro del sistema.

Para el manejo de errores se implementó un paquete exceptions que contiene excepciones personalizadas adaptadas a las necesidades del proyecto. Esto permite detectar situaciones inválidas de manera más clara y proporcionar mensajes de error más específicos durante la ejecución de la aplicación.

También, se incorporó el paquete interfaces, donde se definió la interfaz Calculable. La utilización de interfaces permite establecer contratos de comportamiento que pueden ser implementados por distintas clases, favoreciendo la extensibilidad y el desacoplamiento del sistema.

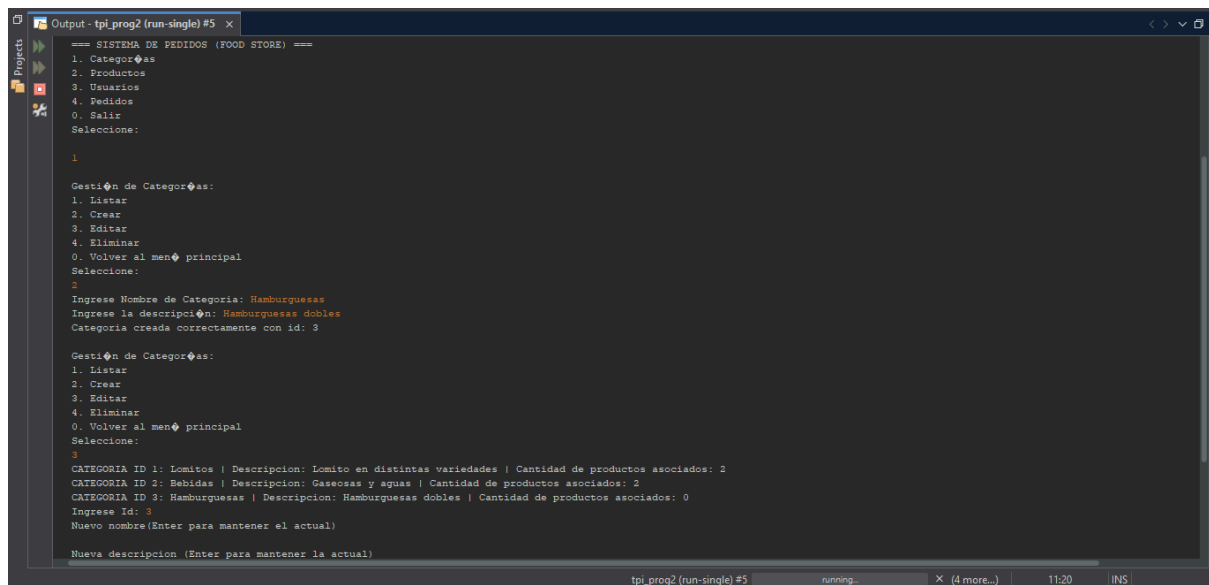
Finalmente, se creó el paquete utils para alojar funcionalidades reutilizables que no pertenecen directamente a ninguna entidad o servicio específico. Contiene la clase Validaciones, la cual centraliza verificaciones comunes como validación de cadenas vacías, comprobación de valores numéricos y otras reglas utilizadas en distintos módulos. Esta decisión permitió evitar la duplicación de código y simplificar el mantenimiento de las validaciones.

En conjunto, esta arquitectura busca mantener una clara separación de responsabilidades entre la presentación, la lógica de negocio, los datos y las entidades del sistema. Esto facilita la comprensión del código, mejora su organización y permite incorporar nuevas funcionalidades con un menor impacto sobre los componentes ya desarrollados.

Además, esta estructura resultó beneficiosa para el trabajo colaborativo del equipo, al encontrarse el proyecto dividido en módulos y clases con responsabilidades bien definidas, cada integrante pudo desarrollar una parte específica del sistema de forma relativamente independiente, reduciendo conflictos y dependencias innecesarias entre los distintos componentes. Esto permitió avanzar de manera paralela en varias funcionalidades y favoreció una mejor coordinación durante el desarrollo del trabajo.

Capturas del sistema:

Menú principal y Gestion de categorias Creación y edición de categorías.



```

Output - tpi_prog2 (run-single) #5 x
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:

1
Gestión de Categorías:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:

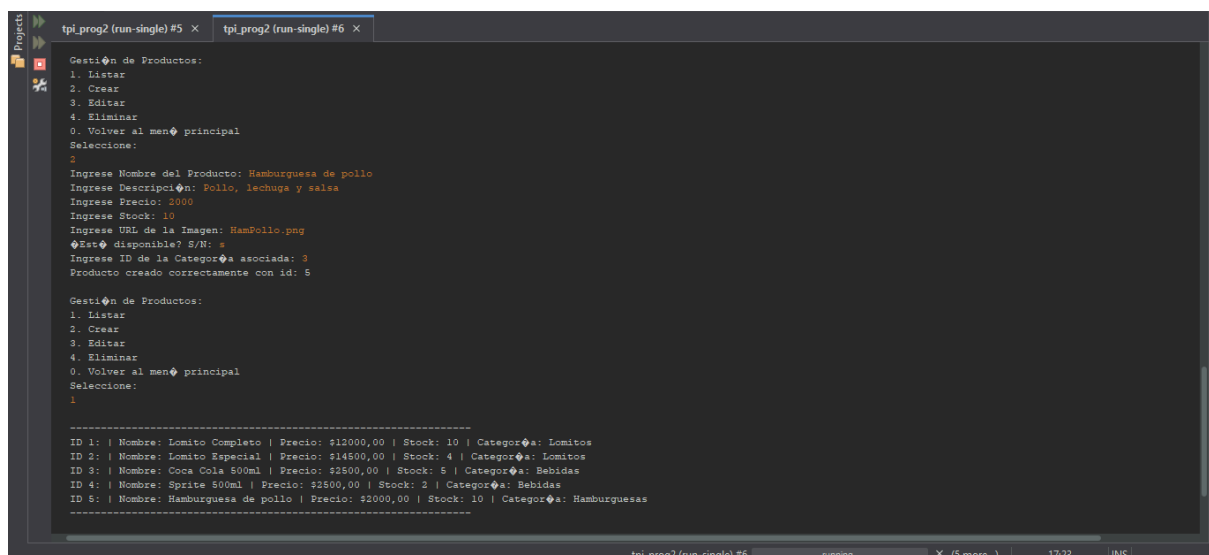
2
Ingrese Nombre de Categoría: Hamburguesas
Ingrese la descripción: Hamburguesas dobles
Categoría creada correctamente con id: 3

Gestión de Categorías:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:

3
CATEGORIA ID 1: Lomitos | Descripción: Lomito en distintas variedades | Cantidad de productos asociados: 2
CATEGORIA ID 2: Bebidas | Descripción: Gaseosas y aguas | Cantidad de productos asociados: 2
CATEGORIA ID 3: Hamburguesas | Descripción: Hamburguesas dobles | Cantidad de productos asociados: 0
Ingrese ID: 3
Nuevo nombre (Enter para mantener el actual)

Nueva descripción (Enter para mantener la actual)
    
```

Gestión de productos. Creación de producto asociado a la categoría creada y listado de productos.



```

tpi_prog2 (run-single) #5 x  tpi_prog2 (run-single) #6 x
Gestión de Productos:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:

2
Ingrese Nombre del Producto: Hamburguesa de pollo
Ingrese Descripción: Pollo, lechuga y salsa
Ingrese Precio: 2000
Ingrese Stock: 10
Ingrese URL de la Imagen: HamPollo.png
¿Está disponible? S/N: s
Ingrese ID de la Categoría asociada: 3
Producto creado correctamente con id: 5

Gestión de Productos:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:

1
-----
ID 1: | Nombre: Lomito Completo | Precio: $12000,00 | Stock: 10 | Categoría: Lomitos
ID 2: | Nombre: Lomito Especial | Precio: $14500,00 | Stock: 4 | Categoría: Lomitos
ID 3: | Nombre: Coca Cola 500ml | Precio: $2500,00 | Stock: 5 | Categoría: Bebidas
ID 4: | Nombre: Sprite 500ml | Precio: $2500,00 | Stock: 2 | Categoría: Bebidas
ID 5: | Nombre: Hamburguesa de pollo | Precio: $2000,00 | Stock: 10 | Categoría: Hamburguesas
-----
    
```

Gestión de usuario. Creación de usuario y correcta eliminación de usuario mediante id asociado.

```

tpr_prog2 (run-single) #5 x  tpr_prog2 (run-single) #6 x
Gestión de Usuarios:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Selección:
2
Ingresar Nombre: Federico
Ingresar Apellido: Gonzalez
Ingresar Mail: fede.gon@gmail.com
Ingresar Celular: 11343442
Ingresar Contraseña: federico
Selección Rol (1. ADMIN, 2. USUARIO): 1
Usuario creado correctamente con id: 3

Gestión de Usuarios:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Selección:
4

-----
ID: 1 | Nombre: Juan | Apellido: Perez | Mail: juan@gmail.com | Rol: USUARIO
ID: 2 | Nombre: Maria | Apellido: Gomez | Mail: maria@gmail.com | Rol: ADMIN
ID: 3 | Nombre: Federico | Apellido: Gonzalez | Mail: fede.gon@gmail.com | Rol: ADMIN
-----

Ingresar ID: 3
¿Está seguro desea eliminar este usuario? S/N: s
Usuario eliminado correctamente
  
```

Gestión de pedidos y detalles. Se elige ID usuario y producto. Se crea correctamente el pedido con su total.

```

tpr_prog2 (run-single) #5 x  tpr_prog2 (run-single) #6 x
Gestión de Pedidos y Detalles:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Selección:
2

Listado de Usuarios Activos:
ID: 1 | Nombre: Juan | Apellido: Perez | Mail: juan@gmail.com | Rol: USUARIO
ID: 3 | Nombre: Federico | Apellido: Gonzalez | Mail: fede.gon@gmail.com | Rol: ADMIN

Para crear un pedido debe ingresar el ID del usuario.
Ingresar ID: 1
Usuario encontrado!
Ingresar ID del producto:
2
Ingresar la cantidad:
2
Total del pedido: $29000.0
Desea agregar otro producto? (S/N)
n
Ingresar ID del producto:
1
Ingresar la cantidad:
1
Total del pedido: $41000.0
Desea agregar otro producto? (S/N)
n
Total del pedido: $41000.0

Pedido creado correctamente.
ID Pedido: 4 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $41000.00 | Forma de Pago: EFECTIVO
  
```

Listado de pedidos y correcta edición en el estado de pedido.

```

Project
  tpi_prog2 (run-single) #5 X
  tpi_prog2 (run-single) #6 X
3
Listado de pedidos:
ID Pedido: 1 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $29000,00 | Forma de Pago: EFECTIVO
ID Pedido: 2 | Usuario: Maria Gomez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $19500,00 | Forma de Pago: EFECTIVO
ID Pedido: 3 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $14500,00 | Forma de Pago: EFECTIVO
ID Pedido: 4 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $41000,00 | Forma de Pago: EFECTIVO

Para actualizar un pedido debe ingresar el ID del pedido.
Ingrese Id: 3

Para actualizar:
1. Estado del Pedido
2. Forma de Pago

Seleccione una opcion:
1

ESTADOS:
1. Pendiente
2. Confirmado
3. Terminado
4. Cancelado

Seleccione un estado:
3

Pedido actualizado correctamente.
ID Pedido: 3 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: TERMINADO | Total: $14500,00 | Forma de Pago: EFECTIVO

Gestión de Pedidos y Detalles:
1. Listar
2. Crear
  
```

Listado de pedidos actualizado con cambio de estado en pedido id: 3 y cambio en forma de pago pedido id: 2.

```

Project
  tpi_prog2 (run-single) #5 X
  tpi_prog2 (run-single) #6 X
1. Estado del Pedido
2. Forma de Pago
Seleccione una opcion:
2
FORMAS DE PAGO:
1. Tarjeta
2. Transferencia
3. Efectivo
Seleccione una forma de pago:
1
Pedido actualizado correctamente.
ID Pedido: 2 | Usuario: Maria Gomez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $19500,00 | Forma de Pago: TARJETA
Gestión de Pedidos y Detalles:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:
1
-----
ID Pedido: 1 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $29000,00 | Forma de Pago: EFECTIVO
ID Pedido: 2 | Usuario: Maria Gomez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $19500,00 | Forma de Pago: TARJETA
ID Pedido: 3 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: TERMINADO | Total: $14500,00 | Forma de Pago: EFECTIVO
ID Pedido: 4 | Usuario: Juan Perez | Fecha: 2026-06-20 | Estado: PENDIENTE | Total: $41000,00 | Forma de Pago: EFECTIVO
  
```

Manejo de errores al crear un pedido con id invalido y al ingresar espacio en blanco.

```
Project
  tpi_prog2 (run-single) #5 x
  tpi_prog2 (run-single) #6 x
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
4
Gestión de Pedidos y Detalles:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:
2
Listado de Usuarios Activos:
ID: 1 | Nombre: Juan | Apellido: Perez | Mail: juan@gmail.com | Rol: USUARIO
ID: 2 | Nombre: Federico | Apellido: Gonzalez | Mail: fede.gon@gmail.com | Rol: ADMIN
Para crear un pedido debe ingresar el ID del usuario.
Ingrese ID: 5
Usuario no encontrado.
Gestión de Pedidos y Detalles:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:
Opción inválida.
```

Manejo de errores al crear producto con precio negativo y al ingresar un id invalido.

```
Project
  tpi_prog2 (run-single) #5 x
  tpi_prog2 (run-single) #6 x
Gestión de Productos:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:
2
Ingrese Nombre del Producto: panchos
Ingrese Descripción: completo
Ingrese Precio: -500
Ingrese Stock: 3
Ingrese URL de la Imagen: pancho.png
¿Está disponible? S/N: s
Ingrese ID de la Categoría asociada: 1
Error: El precio no puede ser menor a 0.
Gestión de Productos:
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:
2
Ingrese Nombre del Producto: pancho
Ingrese Descripción: completo
Ingrese Precio: 2500
Ingrese Stock: 5
Ingrese URL de la Imagen: pancho.png
¿Está disponible? S/N: s
Ingrese ID de la Categoría asociada: 9
Error: La categoría con ID 9 no existe o fue eliminada.
```

Dificultades y Soluciones

Una de las principales dificultades que encontramos durante el desarrollo fue definir cómo organizar correctamente las responsabilidades de cada clase.

En las primeras etapas del proyecto surgió la duda de dónde debía ubicarse determinada lógica ya que algunas validaciones, búsquedas y operaciones podrían implementarse tanto dentro de los menús como dentro de las entidades.

A medida que avanzamos y analizamos la arquitectura del sistema, comprendimos que era conveniente separar claramente cada responsabilidad. Los menús debían encargarse únicamente de interactuar con el usuario, solicitar datos y mostrar resultados, mientras que las clases de servicio debían contener la lógica del negocio y las operaciones sobre las entidades. Por su parte, las entidades debían representar únicamente los datos y comportamientos propios de cada objeto.

Para resolver este problema reorganizamos parte del código y centralizamos la lógica dentro de la capa de servicios. Esta decisión permitió reducir la duplicación de código, mejorar la legibilidad del proyecto y facilitar el trabajo en equipo ya que cada integrante podía trabajar sobre módulos específicos sin afectar directamente el trabajo de los demás. Gracias a esta solución obtuvimos una estructura más ordenada y mantenible.

Al tratarse de un proyecto grupal también surgió el desafío de coordinar el trabajo de varios integrantes sobre una misma base de código. Existía el riesgo de que diferentes personas modificaran los mismos archivos o implementaran soluciones incompatibles entre sí.

Para minimizar estos problemas, se definió una estructura de carpetas y responsabilidades desde las primeras etapas del proyecto. Cada integrante pudo enfocarse principalmente en determinados módulos o clases mientras que las interfaces entre componentes se mantuvieron claras y bien definidas. Esto permitió avanzar de forma paralela en distintas funcionalidades y facilitó la integración final del sistema.

Otra dificultad importante fue decidir cómo almacenar y compartir la información entre los distintos módulos del sistema. Como el proyecto no requería el uso de una base de datos, necesitábamos una forma sencilla de mantener disponibles los datos de categorías, productos, usuarios y pedidos durante la ejecución del programa.

La solución consistió en crear una clase Data que centralizara las colecciones utilizadas por todo el sistema mediante atributos estáticos. De esta manera, todos los servicios pueden acceder a la misma información sin necesidad de duplicar estructuras ni generar inconsistencias entre los datos almacenados.

Por último, durante las pruebas detectamos que muchos errores se producían por datos ingresados incorrectamente por el usuario. Por ejemplo, campos vacíos o valores con formatos inválidos. Inicialmente estas validaciones se realizaban en distintos lugares del código, lo que provocaba repeticiones y dificultaba su mantenimiento.

Para resolver este problema desarrollamos una clase de utilidades denominada Validaciones que reúne métodos reutilizables para verificar distintos tipos de datos. Gracias a esta solución fue posible reducir código repetido, simplificar los menús y asegurar que todas las validaciones se realizaran de forma consistente en todo el sistema.

Bibliografía / Webgrafía

- Se utilizó el contenido brindado por la cátedra Programación II de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional.
- Programación Orientada a Objetos (POO): <https://dev.java/learn/oop/>
- Colecciones: <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html>
- Excepciones: <https://dev.java/learn/exceptions/catching-handling/>
<https://dev.java/learn/exceptions/throwing/>
- Genéricos: <https://dev.java/learn/generics/intro/>

Enlaces

Link al video de resolución del proyecto:

- <https://youtu.be/9PxA3Y8ekpk>

Link al repositorio:

- https://github.com/FacundoVillarroel/Trabajo_Practico_Integrador_Programacion_2